Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Resumen 2do Parcial

Apunte teórico-práctico basado en el contenido de clase

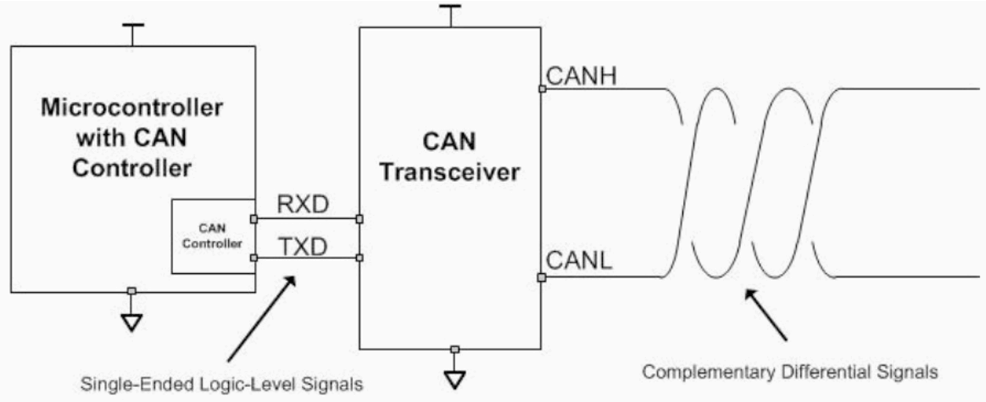
Índice

1 Clase 7 3
1. 1 Protocolo CAN (Controller Area Network) 3
1. 1. 1 Arquitectura y Características Principales 3
1. 1. 2 Capa Física y Señal Diferencial 3
1. 1. 3 Estructura de Trama, Arbitraje y Bit Stuffing 4
1. 1. 3. 1 Trama de Datos Estándar (Standard Data Frame) 4
1. 1. 3. 2 Arbitraje No Destructivo (CSMA/CD + AMP) 4
1. 1. 3. 3 Regla de Bit Stuffing 5
1. 1. 4 Base de Tiempo y Sincronización (Nominal Bit Time) 5
1. 1. 4. 1 División del Bit en Time Quanta t_q 5
1. 1. 5 Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time) 5
1. 1. 6 Filtrado por Hardware y Manejo de Errores 7
1. 1. 6. 1 Filtrado de Mensajes (Filter y Mask) 7
1. 1. 6. 2 Estados del Nodo y Manejo de Errores 7
2 Clase 8 8
2. 1 Periféricos de Temporización y Módulo FlexTimer (FTM) 8
2. 1. 1 Fundamentos y Necesidad de Hardware Dedicado 8
2. 1. 2 Arquitectura del Módulo FlexTimer (FTM) en Kinetis K64F 9
2. 1. 2. 1 Instancias y Distribución de Canales 10
2. 1. 2. 2 Fuentes de Reloj (Clock Source) y Prescaler 10
2. 1. 2. 3 Modos de Conteo del Contador (`FTM Counter`) 11
2. 1. 3 Modos de Operación y Configuración por Canal 13
2. 1. 3. 1 Modo Compatibilidad TPM (`FTMEN = 0`) vs. Modo FTM Avanzado (`FTMEN = 1`) 13
2. 1. 3. 2 Modos de Trabajo Individuales y Combinados 13

2. 1. 3. 3 Filtrado Digital en Captura de Entrada (Input Capture Filter) 14
2. 1. 3. 4 Inserción de Tiempo Muerto (Deadtime Insertion) 14
2. 1. 3. 5 Control de Falla por Hardware (Fault Control) 15
2. 1. 4 Fuentes de Interrupción y Vectores NVIC 15

Clase 7

Protocolo CAN (Controller Area Network)



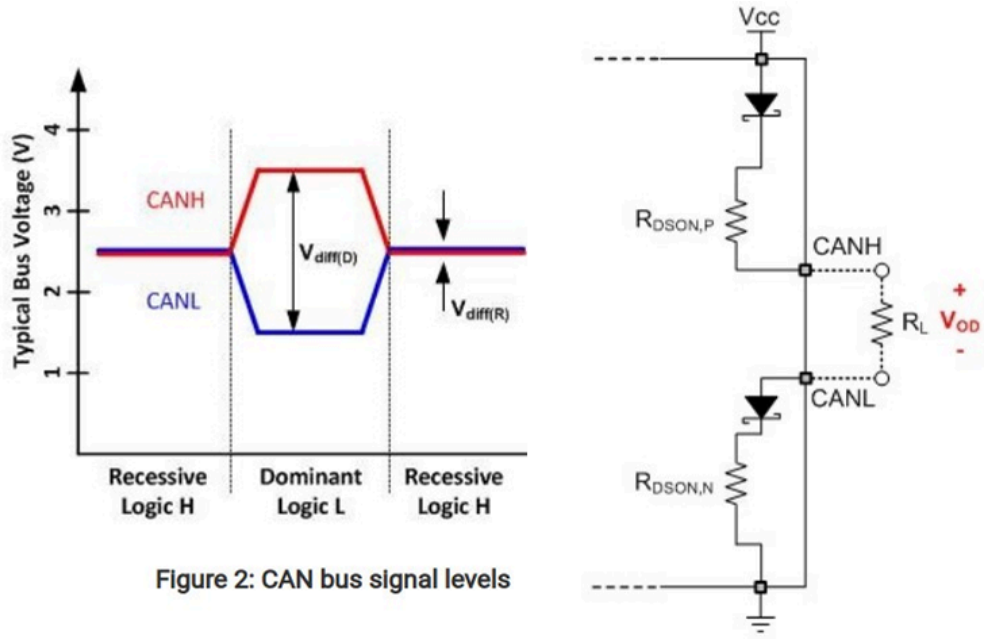


Figure 2: CAN bus signal levels

Arquitectura y Características Principales

CAN es un estándar de bus serie asíncrono, multimaestro y altamente robusto diseñado originalmente por Bosch para aplicaciones automotrices e industriales (ISO 11898). Permite que múltiples Unidades de Control Electrónico (ECUs) se comuniquen sobre un bus común sin necesidad de un nodo maestro central o controlador host.

Características fundamentales del protocolo:

- **Multimaestro / Asíncrono:** Cualquier nodo puede iniciar la transmisión cuando el bus está libre (**Idle**).
- **Orientado a Mensajes:** Los mensajes no contienen direcciones de nodo origen o destino; en su lugar, incorporan un **Identificador (ID)** que define la naturaleza del dato y su prioridad.
- **Inmunidad al Ruido:** Utiliza señalización diferencial sobre par trenzado.
- **Arbitraje No Destructivo:** Resuelve colisiones en tiempo real mediante lógica de bus dominante/recesivo (**Wired-AND**).
- **Detección y Manejo de Errores:** Lógica integrada por hardware para contención y aislamiento de fallas.

Capa Física y Señal Diferencial

A nivel eléctrico, el bus CAN se compone de un par de cables trenzados (**twisted pair**) denominados **CANH** (CAN +) y **CANL** (CAN −), acompañados por una referencia común de masa (GND). Para evitar reflexiones de onda electromagnética en altas frecuencias, el bus exige **resistencias de terminación de 120Ω** instaladas en ambos extremos físicos de la línea.

El bus opera bajo dos estados eléctricos distintos que implementan la lógica **Wired-AND**:

⚙️ Estado Recesivo ('1' Lógico)

- Transceptores en alta impedancia (apagados).
- Las líneas descansan pasivamente a través de resistencias **pull-middle**.
- Voltaje: $V_{\text{CANH}} = 2.5\ \text{V}$, $V_{\text{CANL}} = 2.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = V_{\text{CANH}} - V_{\text{CANL}} = 0\ \text{V}$.

⚙️ Estado Dominante ('0' Lógico)

- Transceptores activos excitando la línea.
- Se fuerzan niveles de tensión separados.
- Voltaje: $V_{\text{CANH}} = 3.5\ \text{V}$, $V_{\text{CANL}} = 1.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = 2.0\ \text{V}$.

💡 Lógica Wired-AND y Colisión

Al ser un bus cableado en configuración **Wired-AND**, si dos nodos intentan transmitir simultáneamente y uno de ellos impone un estado **Dominante ('0')** mientras el otro envía un **Recesivo ('1')**, el estado dominante sobrescribe físicamente al recesivo en todo el bus. Esto permite resolver colisiones sin destruir el mensaje dominante.

Estructura de Trama, Arbitraje y Bit Stuffing

1. 1. 3. 1 Trama de Datos Estándar (Standard Data Frame)

Un **Data Frame** Estándar (CAN 2.0A) se compone de los siguientes campos secuenciales:

Campo	Longitud	Descripción
SOF (Start of Frame)	1 bit	Bit dominante ($1 \rightarrow 0$) que marca el inicio de trama y sincroniza a todos los nodos.
Arbitration Field	12 bits	Contiene el **Identificador (ID)** de 11 bits y el bit **RTR** (0 = Data Frame, 1 = Remote Frame).
Control Field	6 bits	Incluye bits de extensión/reserva (`IDE`, `r0`) y el `DLC` (Data Length Code, 4 bits: 0..8 bytes).
Data Field	0..8 bytes	Carga útil (**payload**) del mensaje (0..64 bits).
CRC Field	16 bits	Secuencia de verificación de redundancia cíclica de 15 bits + 1 bit `CRC Delimiter` (recesivo).
ACK Field	2 bits	`ACK Slot` (1 bit, enviado recesivo por el emisor y forzado a dominante por receptores con CRC válido) + `ACK Delimiter` (1 bit recesivo).
EOF (End of Frame)	7 bits	Fin de trama compuesto por 7 bits recesivos consecutivos.
ITM (Intermission)	3 bits	Espacio de descanso recesivo obligatorio entre tramas (**Interframe Space**).

1. 1. 3. 2 Arbitraje No Destructivo (CSMA/CD + AMP)

Cuando el bus entra en estado pasivo (**Idle**), cualquier nodo con un mensaje pendiente puede emitir el bit `SOF`. Si dos o más nodos inician la transmisión en el mismo instante:

1. Cada nodo transmite los bits de su `ID` mientras monitorea continuamente el nivel físico del bus.
2. Si un nodo envía un bit **recesivo ('1')** en su ID pero lee un nivel **dominante ('0')** en el bus, detecta que otro mensaje en competencia tiene mayor prioridad.
3. El nodo perdedor retira su transmisor inmediatamente, pasa a modo de solo escucha (**listening only**) y aguarda a que el bus vuelva a quedar libre.
4. **Regla de prioridad:** Cuanto **menor sea el valor numérico del ID**, mayor es su prioridad en la red (el bit dominante '0' predomina en las primeras posiciones).

1. 1. 3. 3 Regla de Bit Stuffing

Debido a que CAN es un protocolo asíncrono sin línea física de reloj, los receptores dependen de las transiciones de voltaje ($1 \rightarrow 0$) para sincronizar sus relojes internos.

> ⚠️ **Mecanismo de Bit Stuffing**
>
> Si el transmisor detecta **5 bits consecutivos de la misma polaridad** en el flujo de datos, el hardware inserta automáticamente **1 bit de relleno (Stuff Bit)** de polaridad opuesta antes de enviar el siguiente bit. Los nodos receptores remueven este bit de relleno de forma transparente antes de procesar la trama.
>
> Esta regla se aplica estrictamente desde el bit `SOF` hasta el final de la secuencia `CRC` (se excluyen el `CRC Delimiter`, el campo `ACK` y el `EOF`).

Base de Tiempo y Sincronización (Nominal Bit Time)

1. 1. 4. 1 División del Bit en Time Quanta t_q

En redes CAN, la duración de cada bit (**Nominal Bit Time**) se subdivide en N unidades discretas de tiempo denominadas **Time Quanta** (t_q. La duración de 1 t_q se deriva del reloj interno del microcontrolador (f_{osc} mediante el divisor **Baud Rate Prescaler** (BRP.

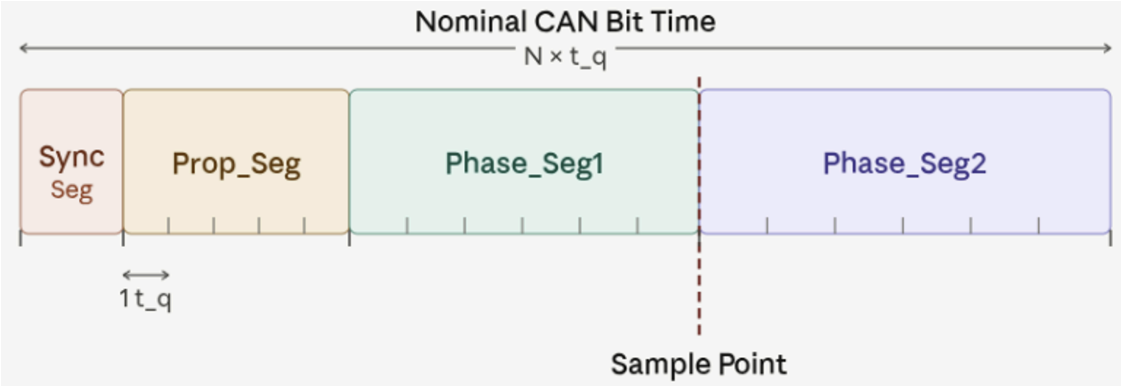


Figura 4: Estructura interna del Nominal Bit Time en sus cuatro segmentos de Time Quanta

El tiempo de bit se divide exactamente en cuatro segmentos configurables:

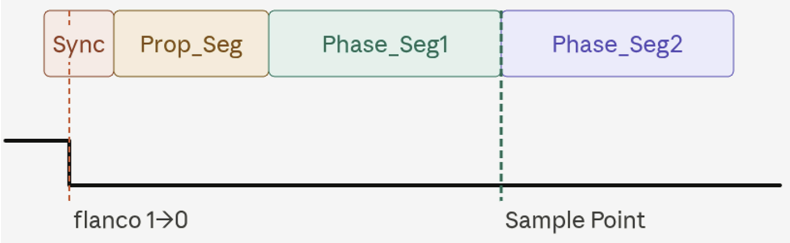
1. **Sync_Seg (Synchronization Segment):** Fijado siempre en 1 t_q. Es el intervalo donde se espera que ocurra el flanco descendente ($1 \rightarrow 0$ que marca el inicio del bit.
2. **Prop_Seg (Propagation Segment):** Configurable de 1 a 8 t_q. Compensa los retrasos físicos de propagación de las ondas electromagnéticas a lo largo del cable y los retardo internos de los transceptores.
3. **Phase_Seg1 (Phase Buffer Segment 1):** Configurable de 1 a 8 t_q. Se ubica inmediatamente antes del **Sample Point**. Se alarga en la resincronización si el flanco llega tarde.
4. **Phase_Seg2 (Phase Buffer Segment 2):** Configurable de 1 a 8 t_q. Se ubica inmediatamente después del **Sample Point**. Se acorta en la resincronización si el flanco llega temprano.

> 💡 **Sample Point (Punto de Muestreo)**
>
> El **Sample Point** es el instante exacto, situado justo al final de `Phase_Seg1`, en el que el nodo evalúa el estado lógico del bus (dominante o recesivo). Se suele configurar típicamente entre el 75% y el 87.5% de la duración total del bit.

Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time)

En el protocolo CAN, la resincronización continua de los nodos receptores se realiza exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$).

Caso 1: Flanco Ideal (Sin Ajuste)



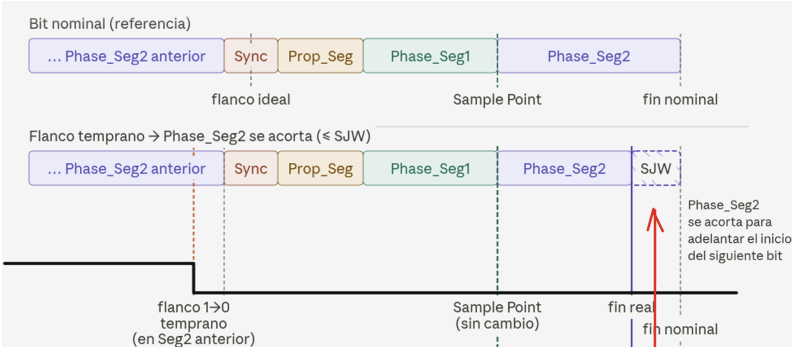
- **Ubicación del Flanco:** El flanco descendente ($1 \rightarrow 0$) llega exactamente dentro del segmento `Sync_Seg` (de duración fija $1t_q$).
- **Estado de Reloj:** El oscilador del nodo receptor está perfectamente sincronizado con el del transmisor.
- **Acción:** No se realiza ningún ajuste en la duración de los segmentos ($\Delta t_q = 0$) y el **Sample Point** permanece en su posición nominal.

Caso 2: Flanco Tardío (Reloj del Transmisor Más Lento)



- **Ubicación del Flanco:** El flanco descendente ($1 \rightarrow 0$) llega con retraso, cayendo en los segmentos `Prop_Seg` o `Phase_Seg1` en lugar de `Sync_Seg`.
- **Estado de Reloj:** El reloj del nodo transmisor es **más lento** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **alarga** el segmento `Phase_Seg1` agregando ciclos t_q adicionales.
  - La cantidad de ciclos agregados es igual al error de fase detectado, acotado por el valor máximo de `SJW` ($+\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Se desplaza el **Sample Point** hacia la derecha (retrasando el momento de muestreo) para capturar correctamente el dato.

Caso 3: Flanco Temprano (Reloj del Transmisor Más Rápido)



- **Ubicación del Flanco:** El flanco descendente ($1 \rightarrow 0$) llega antes de lo esperado, cayendo dentro del segmento `Phase_Seg2` del bit anterior.
- **Estado de Reloj:** El reloj del nodo transmisor es **más rápido** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **acorta** el segmento `Phase_Seg2` substrayendo ciclos t_q de su duración.
  - La reducción está acotada por el valor máximo configurado en `SJW` ($-\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Adelanta la finalización del bit actual e inicia el próximo ciclo antes de lo nominal para reengancharse con el transmisor rápido.

Caso	Ubicación del Flanco	Clock Transmisor	Ajuste de Hardware
Caso 1: Ideal	Dentro de `Sync_Seg`	Sincronizado	Sin ajuste ($\Delta t_q = 0$).

Caso	Ubicación del Flanco	Clock Transmisor	Ajuste de Hardware
Caso 2: Tardío	En `Prop_Seg` / `Phase_Seg1`	Más lento	Se alarga `Phase_Seg1` ($+\Delta t_q \leq \text{SJW}$).
Caso 3: Temprano	En `Phase_Seg2` anterior	Más rápido	Se acorta `Phase_Seg2` ($-\Delta t_q \leq \text{SJW}$).

> ⚠️ **¡Sincronización Solo en Flancos Descendentes!**
>
> En CAN, la sincronización se realiza **exclusivamente sobre los flancos descendentes (1 → 0, paso a dominante)**.
>
> Los flancos ascendentes (0 → 1, paso a recesivo) **no se utilizan** debido a que la subida del bus es pasiva (depende de la resistencia **pull-middle** y capacitancias parásitas del cable), generando pendientes lentas e imprecisas.

Filtrado por Hardware y Manejo de Errores

1. 1. 6. 1 Filtrado de Mensajes (Filter y Mask)

Para evitar sobrecargar la CPU del microcontrolador analizando mensajes no relevantes, el módulo controlador CAN (como el FlexCAN en el Kinetis K64F) filtra las tramas por hardware mediante dos registros:

- **Filter (Identificador de Filtro):** Contiene el patrón binario de ID específico que el nodo está programado para recibir.
- **Mask (Máscara de Aceptación):** Define qué bits del ID deben coincidir obligatoriamente:
  - Bit de máscara en `'1'`: **Verificación estricta** (el bit del ID entrante debe ser idéntico al bit del filtro).
  - Bit de máscara en `'0'`: **Don't Care** (se ignora la coincidencia de ese bit en particular).

1. 1. 6. 2 Estados del Nodo y Manejo de Errores

Cada nodo CAN posee dos contadores internos de error mantenidos por hardware: `TEC` (**Transmit Error Counter**) y `REC` (**Receive Error Counter**). Según sus valores, el nodo transiciona entre tres estados de operación:

Estado del Nodo	Condición de Errores	Comportamiento en la Red
Error Active	$\text{TEC} < 128$ y $\text{REC} < 128$	Operación normal. Al detectar un error, transmite un **Active Error Flag** (6 bits dominantes), abortando la trama de toda la red.
Error Passive	$\text{TEC} \geq 128$ o $\text{REC} \geq 128$	El nodo está degradado. Transmite un **Passive Error Flag** (6 bits recesivos), evitando interrumpir la comunicación de otros nodos.
Bus Off	$\text{TEC} > 255$	El nodo se desconecta lógicamente del bus. No puede transmitir ni recibir hasta reiniciar el módulo FlexCAN.

Clase 8

Periféricos de Temporización y Módulo FlexTimer (FTM)

Fundamentos y Necesidad de Hardware Dedicado

En sistemas embebidos en tiempo real, el uso de interrupciones periódicas por software para generar o medir pulsos presenta limitaciones críticas cuando los anchos de pulso son muy angostos o cuando se requiere una precisión estricta sin sufrir la latencia de atención de la CPU.

Por esta razón, los microcontroladores incorporan bloques periféricos de **Timer por hardware**. Los dos conceptos fundamentales sobre los que se construyen estos periféricos son:

- **Output Compare (OC):** Permite programar un cambio de estado automático en un pin de salida (Toggle, Set, Clear) cuando el contador alcanza un valor predeterminado en el registro de comparación. Este evento ocurre por hardware de forma determinística, eliminando el impacto del jitter e impredecibilidad causados por la latencia de la ISR.

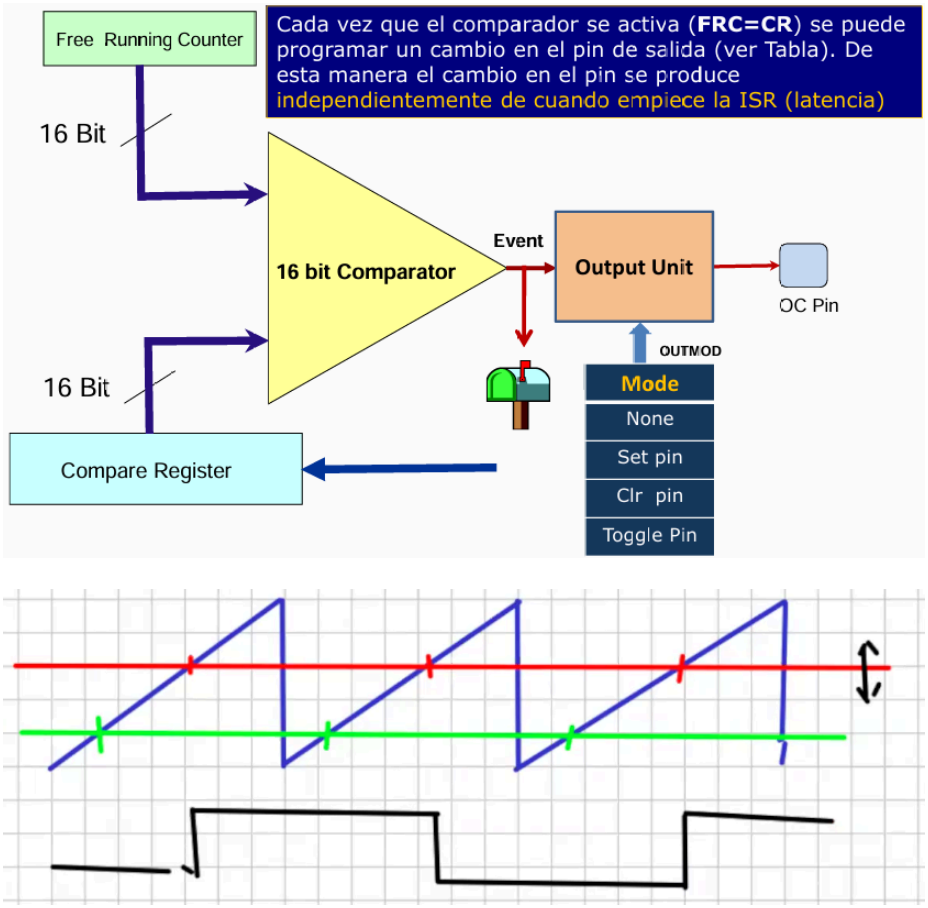


Figura 9: Ejemplo de uso con duty cycle constante

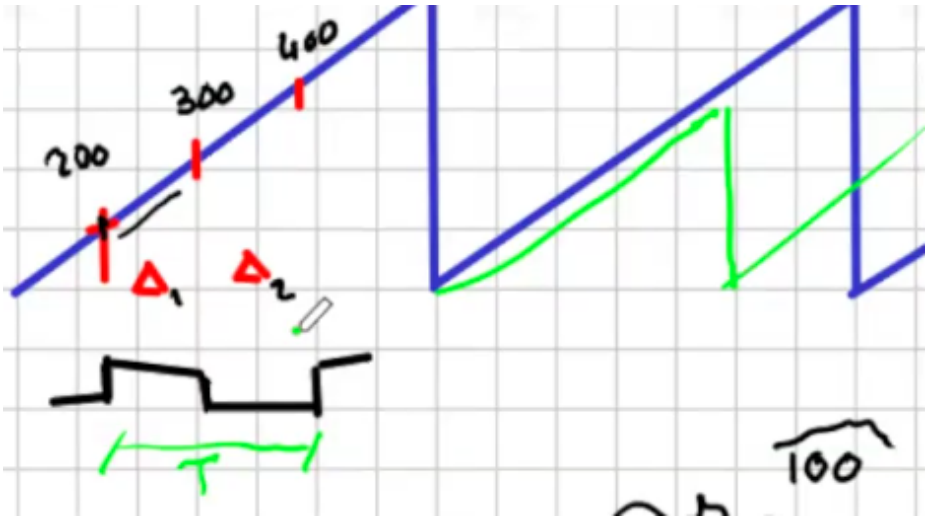


Figura 10: Ejemplo de uso con duty cycle utilizando deltas

- **Input Capture (IC):** Registra (**latch**) el valor instantáneo del contador principal ante la llegada de un flanco en un pin de entrada. Se utiliza para medir frecuencias, períodos o anchos de pulso extremadamente angostos sin sobrecargar la CPU.

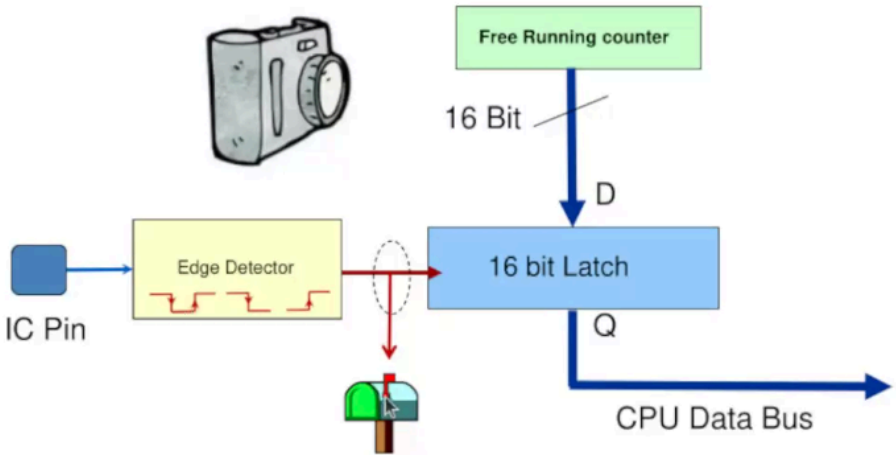


Figura 11: 'Se le saca una foto al contador en el instante del flanco de entrada'

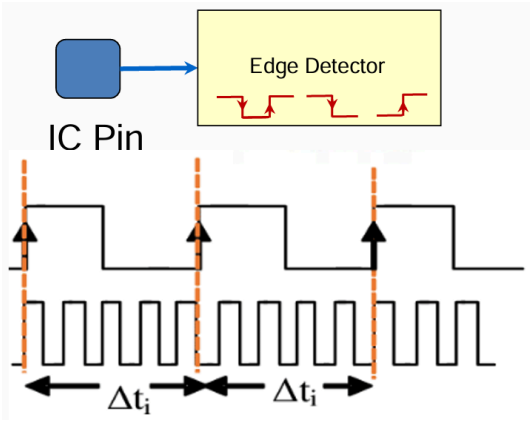


Figura 12: t_i corresponde al clock de FRC (Free Running Counter).)

> **Principio de Funcionamiento por Hardware**
>
> Ambas técnicas se basan en un contador principal de 16 bits (**Free Running Counter / FTM Counter**) accionado por un reloj del sistema [1, 11, 12]. Tanto el latch de captura como la conmutación de los pines de salida se realizan en el mismo ciclo de reloj del evento físico.

Arquitectura del Módulo FlexTimer (FTM) en Kinetis K64F

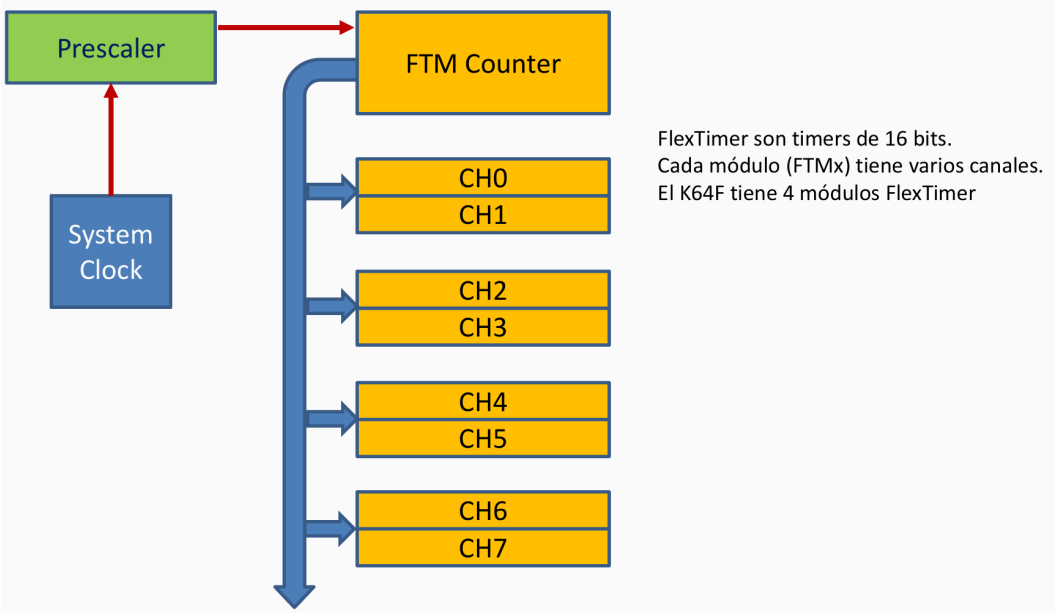


Figura 13: FTM counter y sus-submodulos.

El **FlexTimer Module (FTM)** es un temporizador multicanal de 16 bits integrado en la subfamilia Kinetis K64 (microcontrolador `MK64FN1M0VLL12` montado en la placa FRDM-K64F).

2. 1. 2. 1 Instancias y Distribución de Canales

El microcontrolador K64F dispone de 4 instancias físicas del módulo FTM:

Instancia FTM	N° de Canales	Funciones Principales / Usos
FTM0	8 canales (CH0..CH7)	Control de motores trifásicos, PWM general, motores paso a paso.
FTM1	2 canales (CH0..CH1)	Decodificador en cuadratura (**Quadrature Decoder**) o propósito general.
FTM2	2 canales (CH0..CH1)	Decodificador en cuadratura o propósito general.
FTM3	8 canales (CH0..CH7)	Control de motores trifásicos, PWM y medición de señales.

> ⚙️ **Habilitación del Reloj (Clock Gating)**
>
> Antes de acceder a los registros de cualquier módulo FTM, es obligatorio activar la señal de reloj (**Clock Gate**) en el módulo SIM (**System Integration Module**):
> - **FTM0, FTM1, FTM2:** Se habilitan mediante los bits `SIM_SCGC6_FTM0_MASK`, `SIM_SCGC6_FTM1_MASK`, `SIM_SCGC6_FTM2_MASK` en `SIM->SCGC6`.
> - **FTM2, FTM3:** Se habilitan mediante los bits `SIM_SCGC3_FTM2_MASK` y `SIM_SCGC3_FTM3_MASK` en `SIM->SCGC3`.

2. 1. 2. 2 Fuentes de Reloj (Clock Source) y Prescaler

El contador del FTM recibe su señal de reloj desde una llave de selección configurable en el registro de control `FTMx_SC` mediante el campo `CLKS[1:0]`:

- `00`: Sin reloj seleccionado (deshabilita el contador FTM).
- `01`: **System Clock:** Corresponde internamente al **Bus Clock** del microcontrolador ($f_{\text{bus}} = 50$ MHz en la placa FRDM-K64F a 100 MHz de Core).
- `10`: **Fixed Frequency Clock (`MCGFFCLK`):** Reloj de frecuencia fija independiente.
- `11`: **External Clock (`FTM_CLKINx`):** Reloj desde pines externos seleccionados vía `SIM_SOPT4`.

El valor del reloj seleccionado se divide posteriormente mediante un **Prescaler (PS)** de 3 bits (`PS[2:0]` en `FTMx_SC`), el cual soporta factores de división entre 1 y 128 (2^n con $n = 0..7$):

$$f_{\text{FTM_CNT}} = \frac{f_{\text{Clock Source}}}{\text{Prescaler}}$$

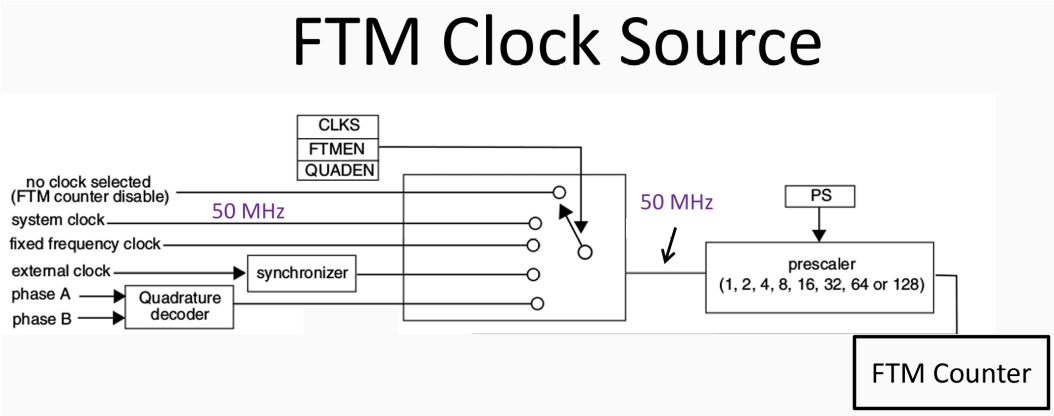


Figura 14: Clock Source

🛠️ Cálculo de Frecuencia y Resolución de Conteo

Si se selecciona el **Bus Clock** (50 MHz) y se configura un prescaler de 4 (`FTM_PSC_x4`): $f_{\text{FTM_CNT}} = \frac{50\text{ MHz}}{4} = 12.5\text{ MHz}$ Resolución de conteo (período de tick) $= \frac{1}{12.5\text{ MHz}} = 80\text{ ns}$

```
void FTM_SetPrescaler (FTM_t ftm, FTM_Prescal_t data)
{
    ftm->SC = (ftm->SC & ~FTM_SC_PS_MASK) | FTM_SC_PS(data);
}
```

En MK64F12.h

```
#define FTM_SC_PS(x)        (((uint32_t)(((uint32_t)(x))<<FTM_SC_PS_SHIFT))&FTM_SC_PS_MASK)
#define FTM_SC_PS_MASK      0x7u
#define FTM_SC_PS_SHIFT     0
```

```
typedef enum
{
FTM_PSC_x1= 0x00,
FTM_PSC_x2= 0x01,
FTM_PSC_x4= 0x02,
FTM_PSC_x8= 0x03,
FTM_PSC_x16= 0x04,
FTM_PSC_x32= 0x05,
FTM_PSC_x64= 0x06,
FTM_PSC_x128= 0x07,
} FTM_Prescal_t;
```

Ejemplo:

```
FTM_SetPrescaler(FTM0, FTM_PSC_x4); // BusClock/2=Sysclk/2= 50MHz y presccaler = divx4
```

Entonces: FTM_Coutner_Clk= 50MHz /4= 12.5MHz (Resolución 80 ns)

2. 1. 2. 3 Modos de Conteo del Contador (`FTM Counter`)

El FTM soporta tres modos de conteo configurables mediante los registros `FTMx_SC` y `FTMx_QDCTRL`:

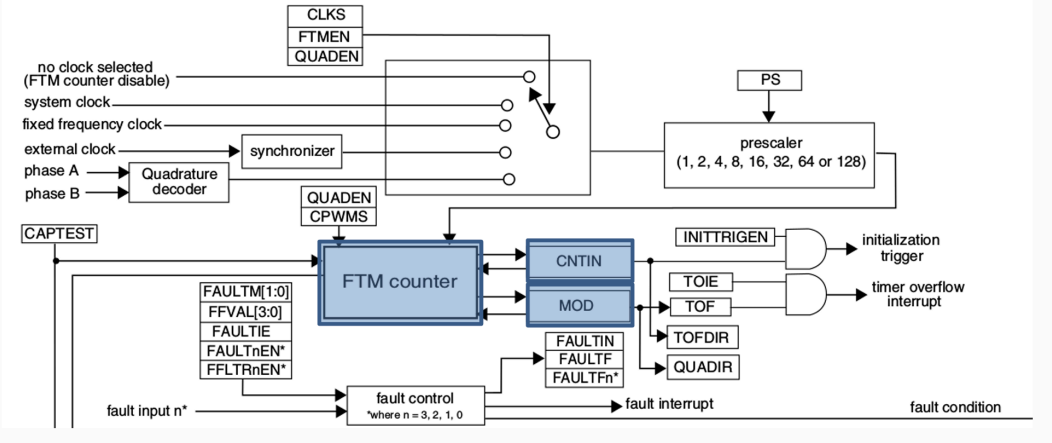


Figura 16: FTM: Up Counting

1. **Up Counting (Conteo Ascendente):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 0`.
   - **Comportamiento:** El contador se carga con `CNTIN` y se incrementa en cada tick hasta alcanzar `MOD`. Al llegar a `MOD`, se recarga automáticamente con `CNTIN` e impone `TOF = 1` (Timer Overflow Flag).
   - **Período de conteo:**

   $$\text{Período} = (\text{MOD} - \text{CNTIN} + 1) \times t_{\text{FTM_CNT}}$$

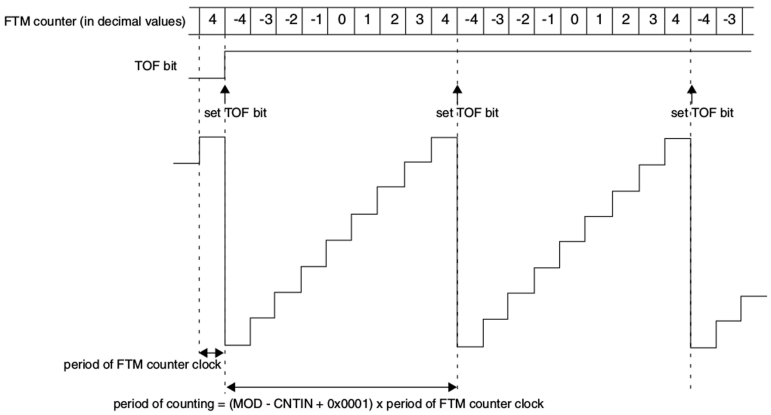


Figura 17: Up Counting, cuenta la cantidad de overflows.

2. **Up-Down Counting (Conteo Ascendente/Descendente - Center Aligned):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 1`.
   - **Comportamiento:** Cuenta desde `CNTIN` hasta `MOD`, y al llegar a `MOD` comienza a decrementar hasta regresar a `CNTIN`. El flag `TOF = 1` se activa al pasar de `MOD` a `MOD - 1`.
   - **Período de conteo:**

   $$\text{Período} = 2 \times (\text{MOD} - \text{CNTIN}) \times t_{\text{FTM_CNT}}$$

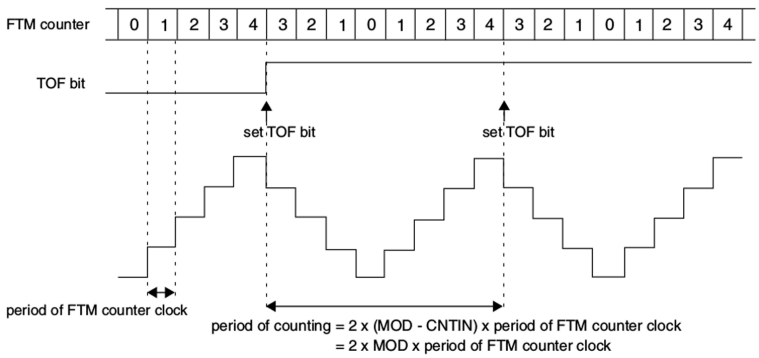


Figura 18: Up-Down Counting

3. **Free Running Counter:**

- **Comportamiento:** Se obtiene fijando `CNTIN = 0x0000` y `MOD = 0xFFFF`. El contador recorre todo el rango de 16 bits (0..65535) continuamente. `TOF = 1` cuando el contador pasa de 0xFFFF to 0x0000

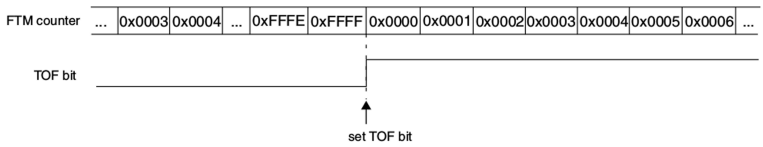


Figura 19: Free Running Counter

> ⚙️ **Registro de Filtro de Overflows (NUMTOF)**
>
> El FTM incluye el registro `NUMTOF[4:0]`, el cual especifica la cantidad de veces que el contador debe sufrir un overflow antes de que el hardware active efectivamente el bit de interrupción `TOF = 1`. Esto permite extender los intervalos de interrupción sin sobrecargar la CPU.

Modos de Operación y Configuración por Canal

2. 1. 3. 1 Modo Compatibilidad TPM (`FTMEN = 0`) vs. Modo FTM Avanzado (`FTMEN = 1`)

El módulo FTM incluye dos modos globales de operación definidos en el registro `FTMx_MODE`:

- **FTMEN = 0 (TPM Legacy):** Modulo compatible con los timers TPM antiguos (arquitectura HCS08). Solo permite el uso del conjunto básico de registros.
- **FTMEN = 1 (FTM Advanced Functions):** Habilita todas las funciones avanzadas: Combine Mode, Deadtime Insertion, Fault Control, Quadrature Decoder, PWM Synchronization e Inversión de Salidas.

> ⚠️ **¡Protección de Escritura (WPDIS)!**
>
> Para escribir en el bit `FTMEN` o en registros avanzados, se debe desactivar primero la protección contra escritura poniendo en `'1'` el bit `WPDIS` (`Write Protection Disable`) en `FTMx_MODE`.

2. 1. 3. 2 Modos de Trabajo Individuales y Combinados

Modo de Canal	Configuración de Bits	Descripción / Comportamiento

Input Capture (IC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 00`
	`ELSnB:ELSnA = 01` (Ascendente)
	`ELSnB:ELSnA = 10` (Descendente)
	`ELSnB:ELSnA = 11` (Ambos flancos)
		Guarda el valor instantáneo del contador `FTM_CNT` en `FTMx_CnV` al detectar el flanco configurado en el pin. Genera interrupción `CHnF = 1`.

Output Compare (OC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 01`
	`ELSnB:ELSnA = 01` (Toggle)
	`ELSnB:ELSnA = 10` (Clear)
	`ELSnB:ELSnA = 11` (Set)
		Compara `FTM_CNT` con `FTMx_CnV`. Al coincidir (`FTM_CNT == CnV`), conmuta la salida del pin automáticamente y activa `CHnF = 1`.

Edge-Aligned PWM (EPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 1X`
	`ELSnB:ELSnA = 10` (High-true)
	`ELSnB:ELSnA = X1` (Low-true)
		PWM alineado al borde con conteo ascendente. La salida cambia al inicio del período y al coincidir con `CnV`.
		Ancho de pulso:
		$$\text{CnV} - \text{CNTIN}$$
		.

Center-Aligned PWM (CPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=1` `ELSnB:ELSnA = 10` (High-true)	PWM centrado con conteo Up-Down. **Ancho de pulso:** $$2 \times (\text{CnV} - \text{CNTIN})$$

Modo de Canal	Configuración de Bits	Descripción / Comportamiento
		. **Período:** $$2 \times (\text{MOD} - \text{CNTIN})$$.
Combine PWM Mode	`FTMEN=1`, `COMBINE=1`, `CPWMS=0` Requiere un par de canales (n par y $n+1$).	El canal n define el borde de encendido (`CnV`) y el canal $n+1$ el borde de apagado (`C(n+1)V`). **Ancho de pulso:** $$\lvert \text{C}(n+1)\text{V} - \text{CnV} \rvert$$.

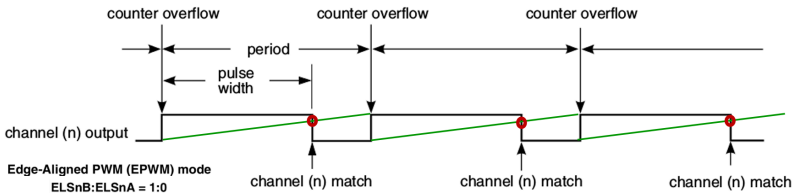


Figura 20: EPWM

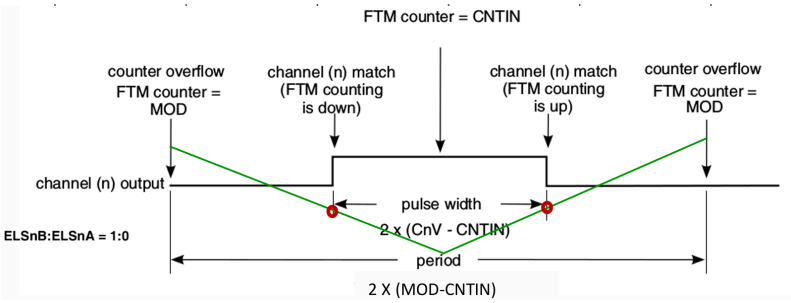


Figura 21: CPWM

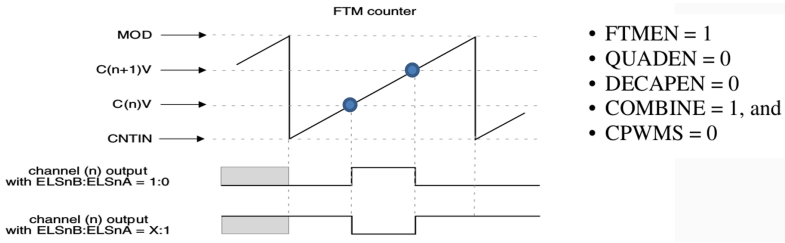


Figura 22: Combine PWM

2. 1. 3. 3 Filtrado Digital en Captura de Entrada (Input Capture Filter)

El FTM incluye filtros digitales por hardware en los canales de entrada $0, 1, 2, 3$ para suprimir ruido o rebotes de señal.

El filtro exige que la señal permanezca estable durante una ventana temporal configurable en `FTMx_FILTER` mediante el campo `CHnFVAL[3:0]`:

Ventana de estabilidad $= 4 + 4 \times \text{CHnFVAL}[3{:}0]$ (en ciclos de clock del sistema)

2. 1. 3. 4 Inserción de Tiempo Muerto (Deadtime Insertion)

En aplicaciones de electrónica de potencia (como el control de motores DC mediante puentes H o inversores trifásicos con MOSFETs/IGBTs), los dos conmutadores de una misma rama nunca deben conducir simultáneamente para evitar un cortocircuito directo (**shoot-through**) de la fuente de alimentación.

El FTM permite configurar salidas complementarias entre los canales n y $n+1$ (`COMP = 1` en `COMBINE`) e insertar de forma automática un tiempo muerto (**Deadtime**) a través del registro `FTMx_DEADTIME`:

⚙️ Registro FTMx_DEADTIME

- `DTPS[1:0]`: Prescaler del reloj de tiempo muerto (divisores 1, 4, 16).
- `DTVAL[5:0]`: Número de ciclos de reloj del prescaler asignados como retardo de conmutación.

2. 1. 3. 5 Control de Falla por Hardware (Fault Control)

El FTM monitorea hasta 4 entradas físicas de falla (`FAULTIN0..3`) para proteger las etapas de potencia. Ante una condición de sobrecorriente o sobretemperatura detectada por hardware, el módulo deshabilita inmediatamente las salidas PWM y las fuerza a un estado seguro sin esperar intervención del software.

Fuentes de Interrupción y Vectores NVIC

Cada módulo FTM consolida sus múltiples eventos de interrupción en **un único vector de interrupción** en el controlador NVIC del microcontrolador:

Fuente de Interrupción	Flags de Habilitación	Registro / Ubicación
Timer Overflow Interrupt	`TOIE = 1` y `TOF = 1`	Registro `FTMx_SC`.
Channel Interrupt (n)	`CHnIE = 1` y `CHnF = 1`	Registro de canal `FTMx_CnSC`.
Fault Interrupt	`FAULTIE = 1` y `FAULTF = 1`	Registro `FTMx_MODE`.

⚙️ Vectores IRQ en el Vector Table de K64F

- `FTM0_IRQHandler` ⟶ Vector 42 en NVIC (`FTM0_IRQn`).
- `FTM1_IRQHandler` ⟶ Vector 43 en NVIC (`FTM1_IRQn`).
- `FTM2_IRQHandler` ⟶ Vector 44 en NVIC (`FTM2_IRQn`).
- `FTM3_IRQHandler` ⟶ Vector 45 en NVIC (`FTM3_IRQn`).